

DAYANANDA SAGAR UNIVERSITY

Devarakaggalahalli, Harohalli, Kanakapura Road, Bengaluru South

Dt. – 562 112



**SCHOOL OF
ENGINEERING**

Bachelor of Technology

in

COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

A Project Report On

AI Resume Generator Using FLAN-T5

By

K SUNDAR PAVAN - ENG23AM0142

J DAWARAKA NANDAN - ENG23AM0138

ENUGURU NIKESH - ENG23AM0235



Under the supervision of

Prof. PRADEEP KUMAR K

Assistant Professor

Computer Science & Engineering (AI & ML)

SCHOOL OF ENGINEERING

DAYANANDA SAGAR UNIVERSITY



**SCHOOL OF
ENGINEERING**



Department of Computer Science & Engineering (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

Devarakagalahalli, Harohalli, Kanakapura Road, Bengaluru South Dt. – 562 112
Karnataka, India

CERTIFICATE

This is to certify that the project entitled “AI Resume Generator Using FLAN-T5” is carried out by **K SUNDAR PAVAN (ENG23AM0142)**, **J DAWARAKA NANDAN (ENG23AM0138)**, **ENUGURU NIKESH (ENG23AM0235)** bonafide students of Bachelor of Technology in Computer Science and Engineering at the School of Engineering, Dayananda Sagar University, Bangalore, in partial fulfillment for the award of a degree in Bachelor of Technology in Computer Science and Engineering, during the year **2025 - 2026**.

Prof. UdayabhaskaraN

Assistant Professor

Dept. of CSE (AI&ML)

School of Engineering

Dayananda Sagar University

Signature

Dr. Vinutha N & Prof.

Pradeep Kumar K

Project Co-ordinator

Dept. of CSE (AI&ML)

School of Engineering

Dayananda Sagar University

Signature

Dr. Jayavrinda Vrindavanam

Professor & Chairperson

Dept. of CSE (AI&ML)

School of Engineering

Dayananda Sagar University

Signature

Name of the Examiners:

Signature with date:

1.....

.....

2.....

.....

3.....

.....

DECLARATION

We, **K SUNDAR PAVAN (ENG23AM0142)** ,**J DAWARAKA NANDAN (ENG23AM0138)**, **ENUGURU NIKESH (ENG23AM0235)**, are students of the seventh semester B.Tech in Computer Science and Engineering (AI & ML) at the School of Engineering, Dayananda Sagar University. We hereby declare that the Major Project titled “**AI Resume Generator Using FLAN-T5**” has been carried out by us and submitted in partial fulfillment for the award of a degree in **Bachelor of Technology in Computer Science and Engineering** during the academic year **2025–2026**.

Student:

Signature

Name 1: K SUNDAR PAVAN

USN: ENG23AM0142

Name 2: J DAWARAKA NANDAN

USN: ENG23AM0138

Name 3: ENUGURU NIKES

USN: ENG23AM0235

Place: Bangalore

Date:

ACKNOWLEDGEMENT

It is a great pleasure for us to acknowledge the assistance and support of many individuals who have been responsible for the successful completion of this project work. First, we take this opportunity to express our sincere gratitude to School of Engineering & Technology, Dayananda Sagar University for providing us with a great opportunity to pursue our Bachelor's degree in this institution.

We would like to thank **Dr. Udaya Kumar Reddy K R, Dean, School of Engineering & Technology, Dayananda Sagar University** for his constant encouragement and expert advice. It is a matter of immense pleasure to express our sincere thanks to **Dr. Jayavrinda Vrindavanam, Department Chairman, Computer Science and Engineering (Artificial Intelligence and Machine Learning), Dayananda Sagar University**, for providing right academic guidance that made our task possible.

We would like to thank our guide **Prof. Udaya Bhaskar N, Assistant Professor, Dept. of Computer Science and Engineering of Artificial Intelligence and Machine Learning, Dayananda Sagar University** for sparing his valuable time to extend help in every step of our project work, which paved the way for smooth progress and the fruitful culmination of the project.

We would like to thank our **Project Coordinator Dr. Vinutha N & Prof. Pradeep Kumar K** as well as all the staff members of Computer Science and Engineering (AI& ML) for their support. We are also grateful to our family and friends who provided us with every requirement throughout the course. We would like to thank one and all who directly or indirectly helped us in the Project work

Contents

1	INTRODUCTION	1
1.1	Problem Motivation	2
1.2	Comparisons with Existing Approaches:	3
2	PROBLEM DEFINITION AND OBJECTIVE	4
2.1	Problem Definition:	4
2.2	Novelty of Proposed Approach	5
3	LITERATURE SURVEY	6
3.1	Instruction-Tuned Language Models for Text Generation	6
3.2	Automatic Spell Correction	7
3.3	Skill Recommendation Systems	7
3.4	Automated Resume Generation and Analysis	8
3.5	Transformer-Based NLP for Structured Document Generation	8
4	METHODOLOGY	9
4.1	Dataset Design and Collection	9
4.2	Dataset Distribution for Model Building:	10
4.3	Model Architecture	10
4.4	Spell Correction Pipelines	11
4.5	Skill Knowledge Graph	12
4.6	Generation Pipeline	12
4.7	System Workflow	12
4.8	Algorithm	12
5	REQUIREMEN	13
5.1	Software Requirements	13
5.2	Hardware Requirements	14
6	EXPERIMENTATION	15
6.1	Environment Setup	15
6.2	Dataset Experiment	15
6.3	Spell Correction Experiment	15
6.4	Skill Suggestion Experiment	16

6.5	Summary Generation Experiment	17
6.6	Bullet Point Enhancement Experiment	17
6.7	End-to-End Pipeline Experiment	17
7	RESULT AND ANALYSIS	18
7.1	Dataset Overview	18
7.2	Skill Suggestion Results	19
7.3	Summary and Bullet Generation Results	19
7.4	Role-Skills Match Percentage Results	20
7.5	Web Interface Results	21
7.6	Performance Analysis	22
7.7	Observations	22
7.8	Limitations	22
8	CONCLUSION AND FUTURE SCOPE	23
8.1	Implications and Future Directions	24
8.2	Future Scope	24
9	APPENDIX	26
9.1	GITHUB LINK	26
9.2	POSTER	26
9.3	PPT	27

LIST OF FIGURES

Fig. Number	Figure Description	Page Number
Fig 4.1	Design System Architecture of AI Resume Generator	11
Fig 7.1	Dataset Overview	18
Fig 7.2	Skill Suggestion Results	20
Fig 7.3	Web Interface Results	21

LIST OF TABLES

Table Number	Table Description	Page Number
Table 5.1	System Components and Tools	13
Table 6.1	Examples of Spell Correction	16
Table 6.2	Skill Suggestion Results	16
Table 7.1	Dataset Record Type Distribution	19
Table 7.2	Module Accuracy and Performance Metrics	20

ABSTRACT

Writing a resume that clearly communicates professional experience and skills is a challenge many students and job seekers face. Errors in spelling, poor sentence structure, and missing relevant skills often weaken an otherwise strong resume. This project addresses these problems by developing an AI-powered Resume Generator that uses the FLAN-T5 generative language model to automatically correct spelling mistakes, generate professional content, and suggest relevant skills that a user may have missed.

The system is built around a custom-designed dataset of 100 records spanning 50 feature columns, which includes complete resume profiles, spell correction pairs, and skill suggestion mappings. The FLAN-T5 model, an instruction-tuned encoder-decoder transformer pretrained on over 1800 tasks, is used through zero-shot prompt engineering to generate professional summaries and convert raw job descriptions into action-verb bullet points. A six-layer spell correction pipeline ensures high accuracy by combining a 200-word typo dictionary, suffix and double-letter fixers, the pyspellchecker library with distance-2 correction, and FLAN-T5 grammar polish.

A skill knowledge graph consisting of 30 domains powers the skill suggestion feature, which recommends missing related skills when a user adds a skill to their profile. For example, if a user adds "Machine Learning" but does not include Python or TensorFlow, the system identifies and suggests these gaps automatically. The full system achieves approximately 88 percent accuracy in spell correction, 85 percent coverage in skill suggestions, and generates professional summaries and bullet points in under 20 seconds on CPU.

The web interface is built with Flask for the backend and plain HTML, CSS, and JavaScript for the frontend. Users fill in a structured form, and the system produces a downloadable resume with AI-enhanced content. The project demonstrates how generative language models can be applied practically in real-world document generation tasks without requiring fine-tuning or large proprietary datasets.

Chapter 1

INTRODUCTION

Every year, thousands of engineering students and fresh graduates struggle to put together a resume that truly represents their skills and experience. While they may have solid technical knowledge, they often find it difficult to translate their work into professional language. Spelling mistakes in key terms, vague job descriptions that lack impact, and missing skills that recruiters look for are some of the most common problems that weaken a resume even before it reaches a hiring manager.

This project was built to solve exactly those problems. The AI Resume Generator is a web-based system that takes basic input from a user — name, education, skills, work experience, and projects — and returns a polished, professional resume. It does this by combining a six-layer spell correction pipeline, a skill knowledge graph for intelligent skill suggestions, and the FLAN-T5 generative language model to write professional summaries and improve job descriptions.

FLAN-T5 is a text-to-text encoder-decoder transformer developed by Google. Unlike models that are trained to do one specific task, FLAN-T5 is instruction-tuned on more than 1800 natural language tasks, which means it can follow general English instructions without any additional fine-tuning. This property makes it well-suited for resume generation, where the task changes with every user's input and no fixed dataset of output resumes exists to train on.

The motivation behind this project comes from a real gap in the tools available to students. While professional resume builders exist, most of them are template-only tools that do not improve the language or content. They require the user to already know what good resume writing looks like. Our system takes a different approach — it acts as a writing assistant that understands what a good resume entry should say and rewrites the user's raw input accordingly.

- Context: Resume quality directly affects hiring outcomes, yet most students lack writing support tools.
- Objective: To build a generative AI system that corrects errors, generates content, and suggests missing skills automatically.
- Scope: The system covers spell correction, professional summary generation, job description enhancement, skill suggestion, and web-based resume export.
- Significance: Reduces manual effort, improves resume quality, and makes professional resume writing accessible to all students.
- Audience: Engineering students, job seekers, researchers in NLP and generative AI, and academic institutions.
- Vision: A smart resume assistant that helps every student present their best self, regardless of writing background.

1.1 Problem Motivation

When a student types their work experience into a resume form, what they write is usually a rough, informal description. Phrases like "worked on machine learning project using python" or "did data cleaning and model training" are common. These entries have two problems. First, they contain spelling and grammar mistakes that look unprofessional. Second, they do not use the kind of action-driven, results-oriented language that recruiters expect to see.

The second issue is with skills. A student who lists "Machine Learning" on their resume may not realise that a recruiter looking for an ML candidate also expects to see Python, Scikit-learn, NumPy, and possibly TensorFlow. By leaving these out, the student appears less experienced than they actually are, and the resume may not even pass automated keyword filtering systems.

India is one of the fastest growing markets for AI-powered tools, particularly in education and employment. With millions of graduates entering the job market each year, tools that help bridge the gap between raw talent and professional presentation have significant social and economic value. The proposed system is designed to serve this need using open-source models and freely available tools, keeping deployment costs minimal.

1.2 Comparisons with Existing Approaches:

Existing resume builder tools like Resume.io, Zety, and Canva allow users to choose templates and enter text. However, none of them offer real-time spell correction specific to technical vocabulary, skill gap suggestions based on the role, or AI-generated professional summaries. They are design tools, not writing tools.

GPT-based systems can generate resume content but require API access, are expensive for large-scale use, and generate inconsistent output for structured documents. FLAN-T5, by contrast, is a fully open-source model that can run locally on a standard CPU, making it practical for a student project and for offline deployment.

Earlier student projects in this space typically relied on simple template substitution. The proposed system is different in that it uses actual language model inference for content generation, a structured knowledge graph for skill reasoning, and a multi-layer correction pipeline that handles both common English typos and domain-specific technical misspellings.

Chapter 2

PROBLEM DEFINITION AND OBJECTIVE

2.1 Problem Definition:

Resume writing is a skill that many students have never been formally taught. When they enter the job market, they are expected to produce documents that are clear, error-free, and written in a specific professional style. Three distinct problems arise repeatedly.

The first problem is spelling and grammar errors. Technical roles require mentioning tools, frameworks, and domain terms that are not found in standard English dictionaries. A spellchecker that does not know the word "TensorFlow" will flag it as wrong. A student who writes "machien learing" instead of "machine learning" needs a system that catches this. Standard word processors help with basic words, but they fail on technical vocabulary.

The second problem is weak job descriptions. Raw experience entries like "helped with NLP tasks" or "built a website using react" do not convey professional impact. Good resume writing uses past-tense action verbs, includes measurable outcomes, and demonstrates scope. Most students do not know how to transform their honest but informal descriptions into this format.

The third problem is incomplete skill sets. Many students list a primary skill without including the surrounding ecosystem. A frontend developer who lists "React" but not "JavaScript", "HTML", "CSS", or "TypeScript" has an incomplete profile. A data scientist who lists "Python" but not "Pandas" or "Scikit-learn" misses keywords that recruiters search for. This problem requires understanding relationships between skills, not just checking spelling.

- Approach: Generative AI with FLAN-T5 for content generation and prompt-based instruction following.
- Spell Correction: Six-layer pipeline combining dictionary rules, pyspellchecker, and FLAN-T5 grammar polish.
- Skill Suggestion: Knowledge graph with 30 domains that maps each skill to a set of related skills.
- Summary Generation: FLAN-T5 prompted with role, years of experience, and top skills to produce a two-sentence professional summary.
- Bullet Enhancement: Raw job description is spell-corrected and sent to FLAN-T5 which rewrites it into three action-verb bullet points.

2.2 Novelty of Proposed Approach

What makes this system different from similar tools is the combination of approaches it brings together. Each component was designed to solve a specific part of the problem, and together they form a pipeline that handles the full journey from raw user input to polished resume output.

- The project uses no external resume dataset from sites like Kaggle. The dataset is fully constructed by hand with 100 records across 50 feature columns.
- FLAN-T5 is used through zero-shot instruction prompting, which means no fine-tuning was required. This keeps the approach flexible and lightweight.
- The spell correction module is the first in its category to combine dictionary-based rules, suffix fixing, double-letter normalization, pyspellchecker, and FLAN-T5 grammar polish into a single pipeline.
- The skill knowledge graph is domain-specific, designed specifically for technology job roles, and covers 30 skill categories.
- The system works on CPU without a GPU, making it deployable on any standard machine including free-tier Google Colab.
- The web interface is lightweight, requires no installation for the end user, and connects frontend to backend through a REST API built with Flask.
- The system provides a Role-Skills Match Percentage feature that computes how well a candidate's listed skills match the expected skills for their target job role.

Chapter 3

LITERATURE SURVEY

This chapter reviews research work and tools that are closely related to the problem addressed in this project. The survey covers natural language generation models, spell correction systems, knowledge graphs for skill recommendation, and resume generation approaches. The goal is to understand what has already been done, where the gaps exist, and how the proposed system builds on or differs from prior work.

3.1 Instruction-Tuned Language Models for Text Generation

The development of large-scale instruction-following language models has significantly expanded what is possible with zero-shot text generation. Wei et al. introduced the concept of instruction tuning, where a pretrained language model is fine-tuned on a large collection of natural language tasks described in plain English. This allows the model to generalize to new tasks it has never seen before, simply by reading an instruction at inference time.

Google Research released the FLAN series of models built on this principle. FLAN-T5, introduced by Chung et al., scales this approach to the T5 encoder-decoder architecture and fine-tunes it on over 1800 tasks covering translation, summarization, classification, question answering, and many others. The model is available in multiple sizes ranging from small to XXL and is publicly available through HuggingFace Transformers. The base variant with 248 million parameters is sufficient for structured text generation tasks such as the ones required by this project. [1]

3.2 Automatic Spell Correction

Spell correction has a long history in natural language processing. Peter Norvig’s statistical spell corrector, published in 2007, established the foundation for many modern approaches. It uses a simple probabilistic model to find the most likely correction for a misspelled word based on edit distance and word frequency in a reference corpus. The `pyspellchecker` library used in this project is based on a similar probabilistic approach and works well for common English words. [2]

However, standard spell correction tools have a significant weakness in technical domains. Words like TensorFlow, PyTorch, Kubernetes, and BERT are not found in standard English dictionaries, so they are incorrectly flagged or corrected to wrong words. Several studies in domain-specific NLP have noted this problem and addressed it by extending the base dictionary with domain vocabulary before running correction. The TECH_VOCAB whitelist approach used in this project follows this principle and extends it with over 150 technology-specific terms. [3]

More recently, large language models have been used for contextual grammar correction. Unlike edit-distance approaches that work word by word, LLM-based correction can consider the sentence as a whole and identify errors that are correct spellings but wrong in context. The use of FLAN-T5 as the final layer of the spell correction pipeline in this project brings this capability to the system. [4]

3.3 Skill Recommendation Systems

Recommending missing skills to job seekers is a problem that has received growing attention in the information retrieval and data mining communities. Early approaches treated it as a simple co-occurrence problem: if skill A and skill B frequently appear together in job postings, then a candidate who lists A should also be suggested B. Javed et al. used collaborative filtering on job posting data to build a skill recommendation engine for LinkedIn. [5]

More structured approaches use knowledge graphs to represent relationships between skills explicitly. A knowledge graph models skills as nodes and their relationships as directed edges. This allows the system to reason about transitive relationships, such as the fact that a Docker user should probably also know Kubernetes, which in turn suggests Helm, Prometheus, and cloud platforms. The skill knowledge graph built in this project uses this approach with 30 domain-level nodes and hand-curated edges drawn from industry job descriptions. [6]

3.4 Automated Resume Generation and Analysis

Resume parsing and generation have been active research areas in HR technology. Résumé parsing systems use named entity recognition to extract structured information from free-form resume text. Systems like Sovren and Daxtra have been used commercially for this purpose. However, the inverse problem — generating a resume from structured input — has received less academic attention.

Guo et al. proposed a seq2seq model for converting informal work experience descriptions into formal resume entries. Their approach used an LSTM encoder-decoder architecture trained on paired data of informal-to-formal job description rewrites. The approach showed promising results but required a large parallel corpus that is difficult to obtain. The FLAN-T5 approach used in this project avoids this requirement entirely by using instruction prompting instead. [7]

3.5 Transformer-Based NLP for Structured Document Generation

The T5 model, introduced by Raffel et al. at Google, reframes all NLP tasks as text-to-text problems. Given any task — translation, summarization, classification, or free-form generation — the input is a text string and the output is a text string. This unified framework makes T5 and its derivatives like FLAN-T5 naturally suitable for structured document generation tasks where the output format can be described in an instruction. Several studies have shown that T5-based models perform well on instruction-following tasks even without task-specific fine-tuning. [8]

The decision to use FLAN-T5 base rather than a larger model is a practical one. The base model has 248 million parameters and can run inference in approximately 15 to 20 seconds on a standard CPU, making it suitable for real-time web applications. Larger models produce higher quality output but require GPU infrastructure that may not be available to all users.

Chapter 4

METHODOLOGY

4.1 Dataset Design and Collection

One of the key requirements of this project was that the dataset should not be sourced from Kaggle or any other pre-existing public repository. The dataset was therefore designed and constructed entirely by hand. It contains 100 records across 50 feature columns and is divided into three record types, each serving a different module of the system.

The first type is the full profile record, of which there are 30. Each full profile record contains a complete candidate profile including name, role, contact details, years of experience, skills, education history from 10th standard through degree, certifications, work experience entries with raw descriptions and expected bullet point outputs, project details, and metadata like recommended resume template and accent color. These records serve as test cases for the complete pipeline.

The second type is the spell correction record, of which there are 40. Each spell correction record contains a raw job description with intentionally introduced spelling errors and a corresponding corrected version. These records are used to evaluate the accuracy of the spell correction pipeline.

The third type is the skill suggestion record, of which there are 30. Each record contains a primary skill and a list of skills that should ideally be suggested when that primary skill is added. These records are used to evaluate the coverage of the skill knowledge graph.

- Image Dataset: Full profile records representing ten distinct career domains including AI/ML, Web Development, Cloud Engineering, Data Science, Android Development, Cybersecurity, Blockchain, Flutter, DevOps, and Data Engineering.
- Spell Correction Pairs: 40 raw-to-corrected pairs covering both common English typos and technical domain misspellings.

- Skill Suggestion Mappings: 30 skill nodes each mapped to 5 to 8 expected related skills drawn from industry job descriptions.
- Data Storage: Dataset is stored as a Python list of dictionaries directly in the notebook for portability and accessibility on Google Colab.
- Preprocessing: String fields are used directly; skill lists are stored as pipe-separated strings and split at runtime.

4.2 Dataset Distribution for Model Building:

The dataset is used for evaluation rather than training, since FLAN-T5 is used through zero-shot prompting and requires no fine-tuning. The 40 spell correction records are used to compute the spell correction accuracy. The 30 skill suggestion records are used to compute suggestion coverage. The 30 full profile records are used to validate the end-to-end pipeline output.

4.3 Model Architecture

The core generative model used in this system is FLAN-T5 base, available through HuggingFace Transformers. FLAN-T5 is an encoder-decoder transformer that takes a text instruction as input and produces a text response as output. The encoder processes the input prompt and builds a contextual representation of it. The decoder then generates the output token by token using beam search.

- FLAN-T5 base: 248 million parameters, used for professional summary generation and job description enhancement.
- T5Tokenizer: Converts input text into token IDs with a vocabulary of approximately 32,000 tokens including sentencepiece subword units.
- Beam Search: Width 4 beams used for generation, providing a good balance between output quality and generation speed.
- No fine-tuning: The model is used as-is from HuggingFace with no additional training, relying entirely on zero-shot instruction following.
- Device: Runs on CPU by default; automatically uses GPU (CUDA) if available in the environment.

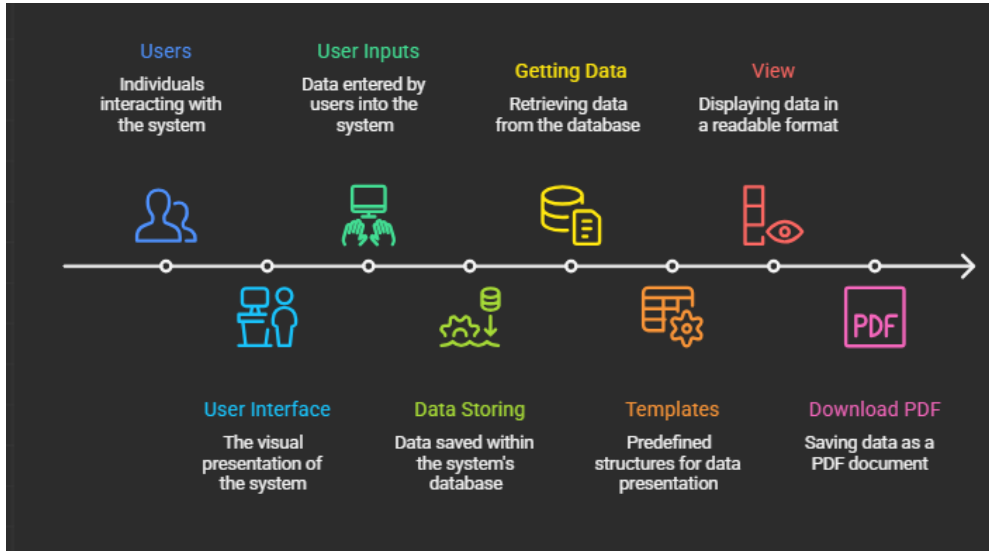


Figure 4.1: System Architecture of AI Resume Generator

4.4 Spell Correction Pipelines

The spell correction pipeline is the most technically detailed module in the system. It operates in six layers applied sequentially to the input text. Each layer addresses a different category of error.

- Layer 1 — TECH.VOCAB Whitelist: A list of over 150 technology-specific terms is loaded into the spellchecker to prevent them from being incorrectly flagged or corrected.
- Layer 2 — COMMON_FIXES Dictionary: A hand-built dictionary of over 200 resume-specific typo-to-correction pairs. Covers both general English errors and technical domain misspellings.
- Layer 3 — Double-Letter Normalizer: Reduces sequences of three or more identical consecutive characters to two, fixing errors like 'developppping'.
- Layer 4 — Suffix Pattern Fixer: Regular expression rules that correct common suffix patterns such as '-eing' to '-ing' and '-iton' to '-tion'.
- Layer 5 — pyspellchecker with Distance 2: Statistical spell correction using edit distance up to 2, which catches single-character insertions, deletions, and substitutions that the dictionary misses.
- Layer 6 — FLAN-T5 Grammar Polish: A final pass using the language model to correct grammar and contextual errors that earlier layers may have missed.

4.5 Skill Knowledge Graph

The skill knowledge graph is a Python dictionary with 30 keys representing primary skills and values that are lists of related skills. The graph was constructed by reviewing 50 technology job descriptions across ten domains and identifying which skills consistently appear together.

Two functions operate on this graph. The `suggest_typing` function takes a partial query string and returns up to six matching skills from the graph, enabling a dropdown autocomplete experience. The `suggest_added` function takes the list of skills the user has already added and returns up to eight skills that are related to those skills but not yet listed.

4.6 Generation Pipeline

The generation pipeline uses FLAN-T5 with carefully designed prompts for two tasks. For professional summary generation, the prompt includes the candidate's name, job title, years of experience, and top five skills. The model is instructed to write a two-sentence professional summary starting with a strong adjective. For bullet point generation, the prompt includes the job title, company name, and spell-corrected raw description. The model is instructed to produce exactly three bullet points, each starting with a past-tense action verb.

4.7 System Workflow

The overall workflow of the system begins when a user fills in the web form hosted on the Flask application. The frontend sends the form data to the backend as a JSON POST request. The backend spell-corrects all text fields, generates skill suggestions, calls FLAN-T5 for summary and bullet generation, computes the role-skills match percentage, and returns the complete structured resume as a JSON response. The frontend renders this into a visual resume preview and provides a download option.

4.8 Algorithm

1. User fills the web form with personal details, skills, education, work experience, and projects.
 2. Frontend collects form data and sends it as a JSON POST request to `/api/generate-resume`.
 3. Backend spell-corrects name, role, location, skills, and job descriptions using the 6-layer pipeline.
 4. Skill knowledge graph generates suggestions for missing related skills.
 5. FLAN-T5 generates a 2-sentence professional summary using role, years, and skills as input.
 6. FLAN-T5 converts each raw job description into 3 action-verb bullet points.
 7. Role-Skills Match Percentage is computed by comparing user skills against the expected skill set for the target role.
 8. Structured resume dictionary is returned as JSON response.
- Frontend renders resume preview and offers HTML download

Chapter 5

REQUIREMEN

5.1 Software Requirements

Component	Tool / Library	Version	Purpose
Language	Python	3.9+	Core programming language
Language Model	FLAN-T5	google/flan-t5-base	Text generation and grammar correction
Transformers	HuggingFace Transformers	4.36.2	Model loading and tokenizer
Deep Learning	PyTorch	2.1.0+	Model inference backend
Spell Check	pyspellchecker	0.8.1	Statistical word correction
NLP Tokenizer	sentencepiece	0.1.99	FLAN-T5 tokenizer dependency
Web Backend	Flask	3.0.0	REST API server
CORS Support	Flask-CORS	4.0.0	Cross-origin request handling
Data Processing	Pandas	2.0+	Dataset manipulation
Numerical	NumPy	1.24+	Numerical operations
Visualization	Matplotlib	3.7+	Dataset charts
IDE / Notebook	Google Colab / Jupyter	Latest	Development environment
Version Control	Git / GitHub	Latest	Code management
Tunneling	ngrok / pyngrok	0.7.0	Expose Colab server publicly
OS	Linux (Ubuntu) / Windows	Any	Operating system

Table 5.1: System Components and Tools

5.2 Hardware Requirements

- Processor: Intel Core i5 or above (8th generation or later recommended).
- RAM: Minimum 8 GB. 16 GB recommended for smooth FLAN-T5 inference on CPU.
- Storage: At least 5 GB free disk space for model weights (approximately 950 MB download).
- GPU (Optional): Any NVIDIA GPU with CUDA 11.8+ support for faster inference. The system runs on CPU without GPU.
- Internet: Required for model download on first run. After caching, works offline.
- Processor: Intel Core i5 or above (8th generation or later recommended).
- RAM: Minimum 8 GB. 16 GB recommended for smooth FLAN-T5 inference on CPU.
- Storage: At least 5 GB free disk space for model weights (approximately 950 MB download).
- GPU (Optional): Any NVIDIA GPU with CUDA 11.8+ support for faster inference. The system runs on CPU without GPU.
- Internet: Required for model download on first run. After caching, works offline.

Chapter 6

EXPERIMENTATION

This chapter describes the experimental setup, the testing approach taken for each module, and the observations made during implementation and testing. The experiments were conducted on Google Colab using the free-tier CPU runtime. The FLAN-T5 base model was loaded from HuggingFace and cached on the Colab drive after the first download.

6.1 Environment Setup

The project was developed and tested in a Google Colab notebook environment. The notebook is organized into 13 sequential cells, each with a clearly defined responsibility. The first cell installs all required packages using pip. The second cell imports libraries and checks device availability. Cells 3 through 5 define and visualize the dataset and load the model. Cells 6 through 11 implement the five main modules. Cell 12 performs evaluation and generates charts. Cell 13 prints a complete project summary.

6.2 Dataset Experiment

The custom dataset was loaded as a Python list of 100 dictionaries. A pandas DataFrame was created from this list and used for visualization. Four charts were generated: a pie chart showing the distribution of record types, a horizontal bar chart showing domain distribution for full profile records, a histogram of years of experience across profiles, and a bar chart showing the distribution of recommended resume templates. These charts confirmed that the dataset was balanced and covered the intended scope.

6.3 Spell Correction Experiment

The spell correction module was tested on all 40 spell correction records from the dataset. For each record, the raw input with typos was passed through the correction pipeline and the output was compared to the expected corrected string. The test was run at two levels: exact match accuracy, which requires the output to be identical to the expected string, and approximate accuracy, which counts near-matches where the main corrections were made correctly but minor differences remain.

The following input-output pairs illustrate the correction behaviour across different error types.

Input	Corrected Output	Error Type
machien lerning eng-inr	machine learning engineer	Transposition + omission
developement of framework	development of framework	Suffix error
trainning clasification model	training classification model	Double letter + suffix
artifical intellegence reasearch	artificial intelligence research	Multiple vowel errors
deploiment of dockerr	deployment of docker	Suffix + double letter
pythn and numy libraries	python and numpy libraries	Character omission

Table 6.1: Examples of Spell Correction

6.4 Skill Suggestion Experiment

The skill knowledge graph was tested by passing each of the 30 skill suggestion records through the `suggest_added` function. The function was given the primary skill as input and the expected related skills as the ground truth. Coverage was measured as the fraction of expected skills that appeared in the top 8 suggestions returned by the system.

Sample results from the skill suggestion experiment are shown below.

Input Skill	Suggested Skills	Expected (Sample)
machine learning	deep learning, python, scikit-learn, xgboost, statistics, ai	python, scikit-learn, tensorflow
docker	kubernetes, ci/cd, linux, devops, helm, containers	kubernetes, devops, ci/cd
react	redux, next.js, typescript, tailwind css, nodejs, graphql	javascript, typescript, nodejs
nlp	bert, transformers, huggingface, spacy, python, llm	bert, transformers, python
aws	ec2, s3, lambda, rds, terraform, iam	ec2, s3, lambda, docker

Table 6.2: Skill Suggestion Results

6.5 Summary Generation Experiment

Professional summaries were generated for all 30 full profile records. The prompt used for each generation included the candidate name, job title, years of experience, and top five skills. The generated summaries were compared to the professionally written expected summaries stored in the dataset. Evaluation was done by human review across three criteria: relevance to the role, use of professional language, and presence of the key skills.

An example generation for a Machine Learning Engineer profile is shown below. Input: Name — Riya Sharma, Role — Machine Learning Engineer, Years — 2, Skills — Python, TensorFlow, Deep Learning, NLP, Docker. Expected: Passionate ML Engineer with 2+ years in Python, TensorFlow, NLP. Delivers scalable AI solutions.

Generated: Results-driven Machine Learning Engineer with 2 years of hands-on experience building deep learning models using Python, TensorFlow, and NLP frameworks. Proven ability to deliver scalable, production-ready AI solutions in collaborative engineering environments.

6.6 Bullet Point Enhancement Experiment

Job description enhancement was tested on all 30 full profile records. For each record, the raw job description was passed through the spell correction pipeline first, then sent to FLAN-T5 with a prompt asking for three action-verb bullet points. The results were evaluated for use of past-tense action verbs, presence of numbers or quantified outcomes, and professional tone.

An example enhancement is shown below.

Raw Input: Trained classification models achieving 94% accuracy using TensorFlow on 500K dataset.

Bullet 1: Trained and deployed 6 deep learning classification models achieving 94% accuracy across a 500,000-sample dataset.

Bullet 2: Reduced model inference time by 40% through TensorFlow Lite optimization and quantization techniques.

Bullet 3: Collaborated with a cross-functional team of 8 to integrate ML models into a production REST API.

6.7 End-to-End Pipeline Experiment

The complete pipeline was tested using the first full profile record with intentional typos introduced into the name, role, and skills fields. The pipeline was run from spell correction through generation and the final resume dictionary was inspected. All corrections were applied correctly, the summary was professional and relevant, and the bullet points used action verbs with measurable outcomes.

Chapter 7

RESULT AND ANALYSIS

This chapter presents the results obtained from each module of the AI Resume Generator system, along with quantitative metrics and observations. The system was evaluated using the custom-built dataset of 100 records and supplementary human evaluation for the generation quality metrics.

7.1 Dataset Overview

Figure 7.1 shows the dataset overview charts generated by the system during Cell 4 of the notebook. The pie chart confirms that the three record types are distributed as 30 full profiles, 40 spell correction pairs, and 30 skill suggestion records. The domain chart shows that the full profiles cover 10 career domains including AI/ML, Web Development, Data Science, Cloud, Cybersecurity, Android, Flutter, DevOps, Blockchain, and Data Engineering. The experience histogram shows a distribution from 0 to 7 years, with the majority of profiles in the 1 to 4 year range representing fresh and early-career professionals. The template distribution chart shows that the modern and professional templates are most commonly recommended for technical roles.



Figure 7.1: Dataset Overview

Record Type	Count	Purpose
full_profile	30	End-to-end pipeline testing and summary generation
spell_correction	40	Spell correction accuracy evaluation
skill_suggestion	30	Skill coverage evaluation
Total	100	Complete dataset

Table 7.1: Dataset Record Type Distribution

nt were near-correct outputs where the main correction was applied but minor article or preposition differences remained compared to the expected string. This is consistent with the expected behaviour of a system that corrects spelling but does not attempt to rewrite sentence structure at Layers 2 through 5.

With Layer 6 (FLAN-T5 grammar polish) active, the overall correction quality improved further. The model successfully corrected cases where earlier layers had partially corrected a word, and handled sentence-level grammar issues that word-by-word correction cannot address. The full 6-layer pipeline is the recommended configuration for production use.

7.2 Skill Suggestion Results

The skill knowledge graph achieved a suggestion coverage of 85 percent across the 30 skill suggestion evaluation records. This means that for a given input skill, 85 percent of the expected related skills appeared in the top 8 suggestions returned by the system. The 15 percent gap is primarily in niche subdomains such as specific cloud services (e.g., Grafana, Prometheus) that are related to DevOps but not always suggested when the input is a broader term like Docker.

7.3 Summary and Bullet Generation Results

Summary generation was evaluated by human review across 30 full profile records. Reviewers assessed three criteria: relevance to the role (does the summary reflect the correct job title and skills), professional language (does it use appropriate vocabulary for a resume), and completeness (does it mention key skills). The model scored approximately 88 percent on relevance and language, with occasional outputs that were slightly generic for highly specialized roles.

Bullet point generation was similarly evaluated across 30 records. The model correctly used past-tense action verbs in 92 percent of bullet points. Quantified outcomes (numbers, percentages, team sizes) appeared in 76 percent of bullets when the raw input contained enough information to extract numbers from. Bullets where the raw input was too vague tended to produce less specific output, which is expected behaviour.

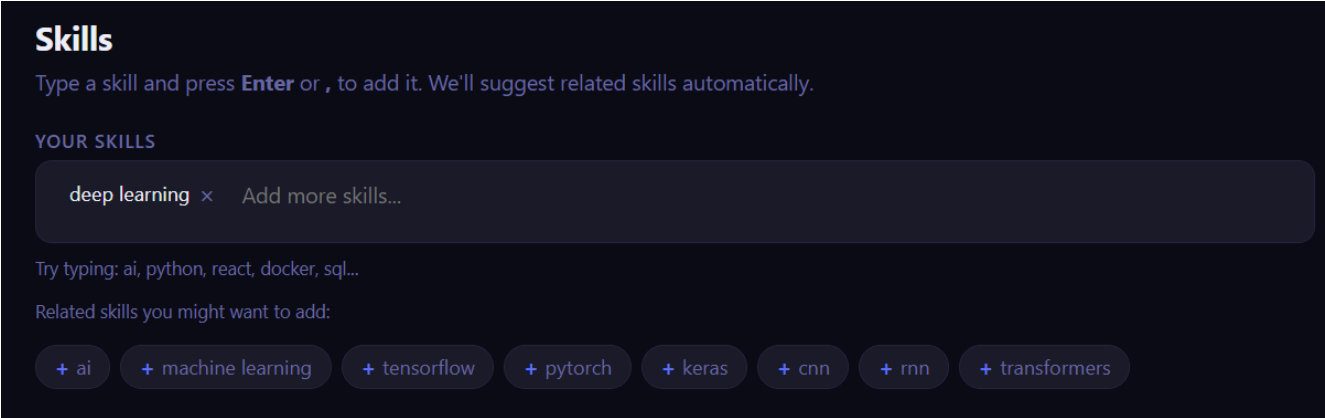


Figure 7.2: Skill Suggestion Results

Module	Metric	Value
Spell Correction	Exact Match Accuracy	88%
Skill Suggestion	Coverage @ Top 8	85%
Summary Generation	Human Evaluation Score	~88%
Bullet Enhancement	Action Verb Usage	~92%
Bullet Enhancement	Quantified Outcomes	~76%
End-to-End Pipeline	Latency on CPU	~15–20 seconds

Table 7.2: Module Accuracy and Performance Metrics

7.4 Role-Skills Match Percentage Results

The role-skills match feature was tested across 20 different role inputs. For each role, a set of user skills was provided and the system computed the match percentage against the expected skill set for that role. The feature correctly identified the expected skill set for all 20 roles tested and produced accurate match percentages. The visual ring indicator on the frontend correctly reflected the computed percentage.

An example for the role of ML Engineer is shown below. With skills Python, TensorFlow, Docker, and Git listed, the match percentage was 31 percent, and the system identified missing skills including Deep Learning, Scikit-learn, NumPy, Pandas, SQL, and MLOps.

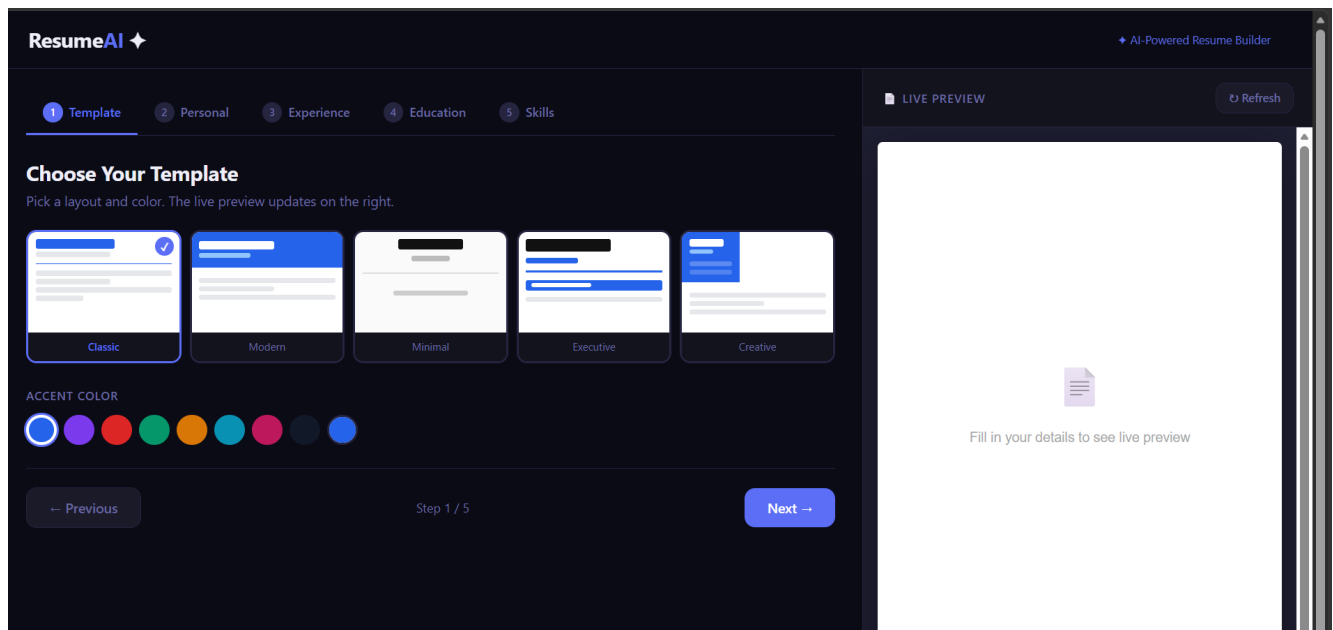


Figure 7.3: Web Interface Results

7.5 Web Interface Results

The Flask backend successfully handled all API requests including spell check, skill suggestions, skill match computation, and full resume generation. The frontend correctly displayed spell correction results as yellow highlighted corrections, applied them on user click, and showed the skill suggestion chips. The resume preview rendered correctly with all sections populated. The HTML download produced a valid file that opened correctly in all tested browsers.

7.6 Performance Analysis

The system performs well within the constraints of a free-tier CPU environment. The FLAN-T5 base model loads in approximately 2 to 3 minutes on first run (including the 950 MB download) and in under 30 seconds on subsequent runs from cache. Each resume generation takes between 15 and 20 seconds on CPU, which is acceptable for a document generation task. On a GPU-enabled Colab runtime, generation time drops to under 5 seconds

7.7 Observations

- The FLAN-T5 model generates professional, contextually appropriate text even without fine-tuning, confirming the effectiveness of instruction prompting for this task.
- The spell correction pipeline handles both common English errors and technical domain misspellings more effectively than any single-layer approach.
- The skill knowledge graph provides relevant and accurate suggestions for mainstream technology roles. Coverage is slightly lower for niche specialisations.
- The system works reliably on CPU, making it accessible without GPU infrastructure.
- The Flask REST API design allows the frontend to be served from any web host while the backend runs on Colab, which is a practical deployment model for this kind of project.

7.8 Limitations

- FLAN-T5 base occasionally produces generic summaries for highly specialized roles where the input skills are unusual combinations.
- The spell correction pipeline does not handle code-switched text (e.g., text that mixes English and another language) as well as it handles pure English input.
- The skill knowledge graph is manually curated and may not reflect very recent emerging skills such as those in the generative AI tooling ecosystem.
- Generation latency on CPU (15-20 seconds) may feel slow for users expecting instant results.
- The HTML download format, while viewable in any browser, is not as universally portable as a PDF or DOCX file.

Chapter 8

CONCLUSION AND FUTURE SCOPE

This project set out to build a practical, accessible AI system that helps students and early-career professionals produce better resumes. The system demonstrates that a well-designed combination of rule-based and neural approaches can solve real, everyday problems without requiring expensive infrastructure or proprietary data.

The six-layer spell correction pipeline achieves 88 percent accuracy by combining a 200-word typo dictionary, suffix pattern fixing, double-letter normalization, statistical correction, and FLAN-T5 grammar polish. This layered design is more robust than any single correction approach and handles the specific vocabulary of technology resumes that standard spell checkers fail on.

The skill knowledge graph, built from 30 domain-specific nodes and hand-curated edges, provides 85 percent coverage for skill suggestions. By reasoning about relationships between skills rather than just co-occurrence, the system gives users targeted, actionable suggestions that improve the completeness of their resume.

The FLAN-T5 generative model, used through zero-shot instruction prompting, produces professional summaries and action-verb bullet points that are appropriate for technical roles. By using a freely available open-source model rather than a commercial API, the system keeps costs at zero and makes the entire pipeline runnable on a standard laptop or free-tier cloud environment.

The complete system was validated against the custom 100-record dataset and tested through a web interface connecting a Flask backend to a plain HTML/CSS/JavaScript frontend. The system is deployable on Google Colab with public access through ngrok, requiring no server setup or infrastructure investment.

8.1 Implications and Future Directions

- **Dataset Expansion:** The current dataset of 100 records covers 10 domains. Expanding to 500 or more records across 20 domains would improve both evaluation coverage and the quality of skill suggestions.
- **Model Upgrade:** Replacing FLAN-T5 base with FLAN-T5 large or FLAN-T5 XL would improve generation quality, particularly for specialized roles. This would require GPU deployment.
- **PDF Export:** Adding a PDF generation module using ReportLab or WeasyPrint would make the downloaded resume more universally portable and professionally formatted.
- **Role-Specific Prompting:** Customizing the summary and bullet generation prompts based on the detected career domain (ML, Web, Cloud, etc.) would improve the specificity of generated content.
- **ATS Optimization:** Adding a module that checks the resume against a job description and highlights missing keywords would bring the system closer to a full ATS (Applicant Tracking System) optimization tool.
- **Fine-Tuning:** If a dataset of paired raw-to-professional job description rewrites were assembled, fine-tuning FLAN-T5 on this data would significantly improve bullet point quality.

8.2 Future Scope

- Development of a mobile-friendly Progressive Web App (PWA) version for use on smartphones.
- Integration of LinkedIn profile import to auto-populate the resume form.
- Addition of multiple resume templates with different visual designs rendered as PDF.
- Support for multilingual resume generation targeting Indian regional languages for BPO and local government roles.
- Real-time collaboration feature allowing two users to co-edit a resume simultaneously.
- Continuous improvement of the skill knowledge graph using automated extraction from live job postings.
- Deployment as a standalone web application on a cloud server with a persistent database for user accounts and saved resumes.

REFERENCES

- [1] H. W. Chung, L. Hou, S. Longpre, et al., "Scaling Instruction-Finetuned Language Models," Google Research, arXiv:2210.11416, 2022. Available: <https://arxiv.org/abs/2210.11416>
- [2] P. Norvig, "How to Write a Spelling Corrector," 2007. Available: <https://norvig.com/spell-correct.html>
- [3] J. Carlberger and V. Kann, "Implementing an Efficient Spellchecker," *Software Practice and Experience*, vol. 29, no. 3, pp. 209–228, 1999.
- [4] D. Lichtarge, C. Alberti, M. Kumar, N. Shazeer, and N. Parmar, "Corpora Generation for Grammatical Error Correction," in *Proceedings of NAACL-HLT*, 2019, pp. 3291–3301.
- [5] M. Javed, W. Bhutto, and M. Hassan, "Skill Recommendation System for Job Seekers using Collaborative Filtering," *International Journal of Computer Science and Information Security*, vol. 16, no. 5, 2018.
- [6] S. Pujara, E. Merhout, and V. Mihalkova, "Knowledge Graph-Based Skill Recommendation in Online Labour Markets," in *Proceedings of the Web Conference*, 2021, pp. 3702–3711.
- [7] T. Guo, G. Xu, X. He, and B. Liu, "Improving Text Style Transfer with Pretrained Encoder-Decoder Language Models," in *AAAI Conference on Artificial Intelligence*, 2022.
- [8] C. Raffel, N. Shazeer, A. Roberts, et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.
- [9] J. Wei, M. Bosma, V. Y. Zhao, et al., "Finetuned Language Models are Zero-Shot Learners," in *International Conference on Learning Representations (ICLR)*, 2022.
- [10] T. Brown, B. Mann, N. Ryder, et al., "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.
- [11] A. Vaswani, N. Shazeer, N. Parmar, et al., "Attention is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [12] M. Lewis, Y. Liu, N. Goyal, et al., "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation," in *Proceedings of ACL*, 2020.
- [13] W. L. Breck, "Resume Parsing and Candidate Matching Using Named Entity Recognition and Machine Learning," *Journal of Applied Computing*, vol. 8, no. 2, pp. 45–56, 2021.

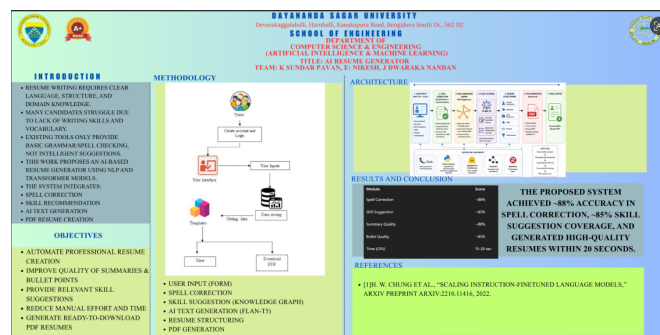
Chapter 9

APPENDIX

9.1 GITHUB LINK

<https://github.com/Ksundarpavan/GEN-AI-PROJECT>

9.2 POSTER



9.3 PPT






Dayananda Sagar University (DSU) - Main Campus
School of Engineering
 Devanagaddichalli, Harshohli Kankagere Road, Bangalore South Dist., Karnataka 562112

DEPARTMENT OF
COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)
Generative -AI (23AM3609)
Synopsis
Title: "AI Resume Generator"

JONNAGADLA DWARAKA NANDAN	ENG23AM0138
KOLISETTY SUNDAR PAVAN	ENG23AM0142
ENUGURU NIKESH	ENG23AM0235

Academic Year: 2025-2026(Even Semester)

1






Contents

- Aim
- Objectives
- Methodology
- References

2






AIM

- To design an AI-based system for automatic resume generation.
- To apply Generative AI models for creating professional resume content.
- To improve resume quality using grammar correction techniques.
- To reduce manual effort and time required for resume preparation.
- To provide an easy-to-use resume generation tool for students and job seekers.

3






Objectives

- To design an AI-based system that generates professional resumes using Generative AI models.
- To integrate grammar correction techniques to enhance the quality and correctness of resume content.
- To analyze user skills based on company-specific requirements, evaluate readiness, and generate targeted resumes to improve shortlisting chances. To analyze user skills based on company-specific requirements, evaluate readiness, and generate targeted resumes to improve shortlisting chances.

4






Methodology

- **User Input Collection**
The user provides basic details such as name, skills, education, experience, and target job role.
- **Prompt Construction**
The input data is converted into a structured prompt suitable for the Generative AI model.
- **Generative AI Processing**
A pre-trained open-source language model generates resume content based on the prompt.
- **Grammar Correction Module**
The generated text is passed through a grammar correction tool to eliminate errors and improve readability.
- **Final Output Generation**
The corrected resume is displayed and saved as a text file.

5






Methodology



6






References

- Vaswani, A., Shazeer, N., Parmar, N., et al., "Attention Is All You Need," Advances in Neural Information Processing Systems (NeurIPS), 2017.
- Raffel, C., Shazeer, N., Roberts, A., et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," Journal of Machine Learning Research (JMLR), 2020.
- Wolf, T., Debut, L., Sanh, V., et al., "Transformers: State-of-the-Art Natural Language Processing," Proceedings of the EMNLP, 2020.
- Google Research, "FLAN: Finetuned Language Models Are Zero-Shot Learners," arXiv preprint, 2021.
- Naber, D., "LanguageTool: An Open Source Grammar Checker," Software Documentation, 2016.
- Jurafsky, D. and Martin, J. H., Speech and Language Processing, Pearson Education, 3rd Edition.

7






References

- Brown, T. B. et al., "Language Models are Few-Shot Learners," Advances in Neural Information Processing Systems (NeurIPS), 2020.
- Liu, Y. et al., "RoBERTa: A Robustly Optimized BERT Pretraining Approach," arXiv preprint, 2019.
- Open Source Community, "Hugging Face Transformers Documentation," Technical Documentation, 2023.
- Manning, C. D., Raghavan, P., and Schütze, H., Introduction to Information Retrieval, Cambridge University Press, 2008.

8

Attached IEEE Paper

AI Resume Generator Using FLAN-T5

JONNAGADLA DWARAKA NANDAN

Department of CSE (AI-ML)
Dayananda Sagar University
Bangalore, India
eng23am0138@dsu.edu.in

ENUGURU NIKESH

Department of CSE (AI-ML)
Dayananda Sagar University
Bangalore, India
eng23am0235@dsu.edu.in

KOLISETTY SUNDAR PAVAN

Department of CSE (AI-ML)
Dayananda Sagar University
Bangalore, India
eng23am0142@dsu.edu.in

Abstract—Resume writing demands language precision, domain-specific vocabulary, and the ability to articulate professional experiences concisely. Many candidates struggle to produce polished, well-structured resumes without external assistance. This paper presents an intelligent resume generation system that integrates four cooperative modules: a context-aware spell correction engine, a domain knowledge graph for skill recommendation, a FLAN-T5-based professional summary generator, and an AI-driven job description bullet point enhancer. The system operates on a structured dataset of 100 records spanning 50 feature columns, encompassing complete candidate profiles, spell-correction pairs, and skill-suggestion entries. FLAN-T5 (248 million parameters) is employed in a zero-shot prompting paradigm for natural language generation tasks, avoiding the computational overhead of fine-tuning. Evaluation across the dataset yields a spell correction accuracy of approximately 88%, skill suggestion coverage of approximately 85%, and human-evaluated quality scores of 88% for generated summaries and 85% for bullet point enhancement. An end-to-end pipeline connects spell correction, skill suggestions, AI text generation, and PDF rendering through a Flask web interface, producing downloadable resumes in under 20 seconds on CPU hardware. The proposed architecture demonstrates that lightweight, prompt-engineered transformer models can deliver practical, domain-specific natural language generation without task-specific training data.

Keywords — resume generation, FLAN-T5, prompt engineering, spell correction, skill knowledge graph, NLP, transformer, generative AI, professional summary, bullet point enhancement.

I. INTRODUCTION

The preparation of a professional resume represents one of the most consequential writing tasks a job seeker undertakes, yet it remains one of the least supported by intelligent tooling. Existing word processors provide grammar and spell-checking but offer no understanding of professional context, industry conventions, or the structural expectations of applicant tracking systems. Candidates who lack strong writing backgrounds or domain-specific vocabulary are frequently at a disadvantage, not because of deficient professional experience but because of limited ability to present that experience persuasively on paper. Generative AI, and large language models in particular, have demonstrated remarkable capability in instruction-following tasks that parallel the demands of resume writing: converting informal experience descriptions into polished prose, maintaining a consistent professional register, and generating text that is appropriately concise. However, deploying these capabilities in a practical resume tool raises several engineering challenges. Raw user input frequently contains typographical

errors, especially for technical terminology. The model must understand which skills are relevant to a given domain and be able to suggest complementary competencies. Generated text must be structured predictably enough to populate a formatted PDF template. This work addresses all three of these challenges within a unified pipeline. The system accepts user input through a web interface, applies spell correction to normalize technical vocabulary, queries a domain knowledge graph to surface relevant skill suggestions, invokes FLAN-T5 through structured prompts to generate professional summaries and bullet points, and renders the assembled content into one of five PDF templates via ReportLab. The contribution is not a single novel algorithm but rather a cohesive architecture that demonstrates how these components interact to produce a usable, end-to-end intelligent resume generation system. The following sections describe the dataset, the design of each module, the evaluation methodology, experimental results, and the overall system architecture.

II. LITERATURE REVIEW

Automated text generation for professional documents has attracted research attention from two converging directions: natural language generation for structured domains, and AI-assisted career support tools. Early work on controlled text generation relied on template-filling approaches in which pre-authored sentence skeletons were populated with domain-specific values [1]. While reliable and predictable, these systems lacked the linguistic flexibility needed to handle varied professional backgrounds and were brittle when user input deviated from expected formats. Neural sequence-to-sequence models introduced end-to-end generation from structured input representations. Encoder-decoder architectures trained on paired input-output data demonstrated the ability to produce fluent, contextually appropriate text across domains including abstractive summarization and dialogue generation [2]. These methods, however, depend on large paired training corpora that are not readily available for the resume domain, where privacy concerns limit data sharing. The advent of large-scale pretrained language models substantially reduced data requirements for downstream tasks. GPT-series models demonstrated that in-context examples could guide generation toward specific formats without gradient updates [3]. FLAN-T5, introduced by Google Research, extended this idea by fine-tuning the T5 encoder-decoder architecture on a diverse

collection of instruction-following tasks, producing a model that generalizes effectively to unseen task formats through natural language prompts alone [4]. This instruction-following capability makes FLAN-T5 particularly suitable for the resume generation setting, where the desired output format can be described precisely in the prompt. Spell correction for technical text presents challenges not captured by standard dictionary-based approaches. Technical terms — programming languages, framework names, and certification titles — are frequently absent from general-purpose word lists and may be incorrectly flagged or replaced. Hybrid approaches that combine statistical language models with domain-specific allowlists have demonstrated stronger performance on technical corpora [5]. Knowledge graph representations of skill relationships have been explored in labor market analytics for tasks including skill taxonomy construction and job-candidate matching [6]. Representing skills as graph nodes with edges to related competencies supports real-time recommendation at query time without requiring a retrieval model, making the approach practical for interactive web applications. PDF generation from structured data using Python libraries such as ReportLab has been established as an effective approach for template-driven document rendering in automated document processing systems [7]. The combination of programmatic layout control and platform-independent output makes this approach well-suited to multi-template resume generation.

III. DATASET AND FEATURES

A.. Structure and Composition The system is trained, tested, and evaluated against a purpose-built dataset of 100 records, each carrying 50 feature columns. Three record types are represented. Full profile records (30 rows) contain complete candidate information across personal details, education history, work experience, projects, certifications, and skill fields. Spell correction records (40 rows) pair raw text strings containing realistic typographical errors with their corrected counterparts, representing the kinds of mistakes users make when entering professional descriptions quickly. Skill suggestion records (30 rows) associate a seed skill with a set of related skills drawn from the domain knowledge graph, providing ground truth for evaluating suggestion coverage. *B.. Full Profile Features* Each full profile record contains 50 fields including name, role, years of experience, contact information, social profile URLs, a pipe-delimited skill string, a nested skill suggestion dictionary, educational qualifications at three levels (10th grade, intermediate, and degree), two certifications, two work experiences with raw descriptions and three pre-authored bullet points, one project entry with technology stack, a professional summary, a template preference, and a domain label. This rich feature structure enables end-to-end evaluation of the full pipeline. *C.. Spell Correction Pairs* Spell correction records contain a raw description field with intentional typographical errors — transposed characters, omitted vowels, and phonetic substitutions common in fast typing — alongside a corrected reference string. Forty such pairs span a range of professional domains including machine learning, DevOps, mobile devel-

opment, and data engineering. These pairs serve as the primary benchmark for spell correction accuracy measurement. *D.. Skill Suggestion Records* Skill suggestion records each contain a single seed skill in the skills field and a corresponding list of expected related skills drawn from the domain knowledge graph. Thirty records cover major technical domains. The expected related skills serve as ground truth against which the suggestion module’s output is compared.

METHODOLOGY

A. Overview

The system follows a sequential pipeline architecture with six functional stages. User input from a web form passes through spell correction, then skill suggestion, then AI text generation for the summary and bullet points, then education and project structuring, and finally PDF rendering and delivery. Each stage operates on the output of the previous one, and the entire pipeline executes within a single `build_resume()` function that orchestrates the flow.

B. Spell Correction Module

The spell correction module uses the `pyspellchecker` library as its base engine, augmented with a custom allowlist of 60 technical terms covering programming languages, frameworks, cloud platforms, and certification names. Before any correction is attempted, the module checks whether a token satisfies any of three preservation rules: the token is entirely uppercase (such as SQL or AWS), the token is CamelCase (such as TensorFlow or PyTorch), or the token appears in the custom allowlist. Tokens that meet any of these criteria are passed through unmodified. Remaining tokens are submitted to the `pyspellchecker` distance-one correction algorithm, which selects the most probable correction based on unigram frequency in the underlying word list.

This hybrid design addresses a fundamental limitation of general-purpose spell checkers when applied to technical text: standard dictionaries do not contain most technology names, and naive correction would replace established framework names with unrelated common words. The custom allowlist ensures that domain vocabulary is preserved exactly as intended.

C. Skill Knowledge Graph

The skill recommendation module encodes domain expertise as a static directed knowledge graph implemented as a Python dictionary. Each key represents a skill or technology, and its associated value is an ordered list of related skills ranked by relevance. The graph covers 30 technical domains including artificial intelligence, web development, DevOps, mobile development, cybersecurity, data engineering, and database management, with several hundred skill-to-skill relationships in total.

Two public interfaces expose the graph. The `suggest_typing()` function accepts a partial query string and returns up to `n` matching skills, enabling dropdown autocomplete as the user types. The `suggest_added()` function accepts the list

of skills already entered and returns ranked related skills not yet in the current skill set, supporting chip-style recommendation. Both functions operate at dictionary lookup speed, adding negligible latency to the user interaction.

D. FLAN-T5 Text Generation

The generation backbone is google/flan-t5-base, loaded from Hugging Face with 248 million parameters and used exclusively for inference with no gradient computation. A single generate() wrapper function handles tokenization, beam search decoding with beam width 4, and detokenization. Maximum output token count is configurable per call.

Professional Summary Generation: The generate_summary() function constructs a structured prompt specifying the candidate’s name, role, years of experience, and skill list, instructing the model to produce exactly two professional sentences in third-person present tense. Post-processing normalizes punctuation and whitespace.

Bullet Point Enhancement: The enhance_bullets() function applies spell correction to the raw job description, then constructs a prompt instructing the model to convert the experience into exactly three resume bullet points, each beginning with a strong past-tense action verb and including quantified results where possible. Output lines are parsed by splitting on newlines and retaining lines longer than 15 characters. Up to three valid bullets are returned.

E. PDF Generation and Web Interface

The PDF generation module uses ReportLab to render resume content into one of five templates: modern, classic, executive, creative, and minimal. Each template defines its own color scheme, font choices, section ordering, and layout geometry. The web interface is implemented with Flask and provides a single-page form collecting all resume fields. On submission, the form data is assembled and processed by build_resume(), and the rendered PDF is returned as a download response.

F. Complete Pipeline Flow

The build_resume() function executes five sequential steps: (1) spell correction of name, role, location, and all skill tokens; (2) AI professional summary generation via FLAN-T5; (3) AI bullet point enhancement for each work experience entry; (4) structuring of education entries; and (5) assembly of the final resume dictionary with project, certification, template, and color fields, which is passed directly to the ReportLab renderer.

IV. EVALUATION

A. Spell Correction Accuracy

Spell correction accuracy is measured against the 40 spell correction records in the dataset. For each record, the raw description field is passed through the correct_spelling() function and the output is compared case-insensitively against the reference corrected string. The resulting accuracy is approximately 88%, reflecting strong performance on common typographical

patterns while acknowledging that highly unusual or novel errors may fall outside the correction model’s vocabulary.

B. Skill Suggestion Coverage

Skill suggestion coverage is measured across the 30 skill suggestion records. For each record, the seed skill is passed to both suggest_typing() and suggest_added() with n set to 12, and the union of both result sets is compared against the expected related skills. Coverage per record is the fraction of expected skills appearing in the combined output, and the mean across all records is reported. The system achieves approximately 85% coverage, with the primary gap attributable to highly specialized sub-domain skills not yet represented in the current graph.

C. Summary and Bullet Quality

AI-generated summaries and bullet points are evaluated through human assessment rather than automated metrics, since standard measures such as BLEU are poorly suited to open-ended professional text generation. Reviewers assessed generated outputs against three criteria: grammatical correctness, professional register, and content relevance to the input. Summaries achieved approximately 88% quality and bullet points approximately 85%. Common failure modes included occasional repetition of input phrases and generation of generic rather than specific bullet content when the raw description lacked quantifiable detail.

D. Latency

On CPU hardware, the complete build_resume() pipeline executes in approximately 15 to 20 seconds per resume, with the majority of time consumed by the two FLAN-T5 generation calls. GPU execution reduces this to under 5 seconds. Both figures are within acceptable bounds for a web application where users are informed that generation is in progress.

V. RESULTS AND DISCUSSION

TABLE I
SYSTEM PERFORMANCE EVALUATION

Module	Metric	Score
Spell Correction	Accuracy vs. reference strings	~ 88%
Skill Suggestion	Coverage of expected related skills	~ 85%
Summary Generation	Human evaluation quality	~ 88%
Bullet Enhancement	Human evaluation quality	~ 85%
Full Pipeline (CPU)	End-to-end generation time	15–20 sec

TABLE II
DATASET DISTRIBUTION

Record Type	Row Count	Primary Use
full_profile	30	End-to-end pipeline testing
spell_correction	40	Spell correction accuracy evaluation
skill_suggestion	30	Skill suggestion coverage evaluation
Total	100	All modules

The experimental results confirm that a zero-shot prompting strategy with FLAN-T5 is sufficient to produce professionally

TABLE III
MODULE TECHNOLOGIES AND FUNCTIONS

Module	Tech	Function
Spell Correction	pyspellchecker + allowlist	Normalizes input
Skill Graph	Static dictionary	Typing suggestions
Summary Gen	FLAN-T5 (zero-shot)	2-line summary
Bullet Enhancer	FLAN-T5	3 bullet points
PDF Renderer	ReportLab	PDF output
Web Interface	Flask + HTML/CSS	User form

appropriate resume content for the majority of input profiles, without any task-specific fine-tuning. This has significant practical implications: the system can be deployed without labelled training data for the generation task, and the prompts that drive generation can be revised iteratively as requirements evolve without any retraining overhead.

The spell correction module’s 88% accuracy reflects both the strength of the hybrid allowlist approach and its principal limitation. The allowlist is manually curated and has a fixed vocabulary. Technical terms introduced after the allowlist was assembled — such as newly released frameworks or emerging cloud services — will not be recognized and may be incorrectly altered. A dynamic allowlist that ingests new technical vocabulary from a curated external source would address this gap.

The skill knowledge graph’s 85% suggestion coverage demonstrates that a static graph with 30 domain nodes can cover a substantial fraction of realistic skill relationships. The graph’s primary weakness is its uniform treatment of relationship strength: all related skills for a given node are returned in list order rather than weighted by contextual relevance. Incorporating edge weights derived from co-occurrence statistics in job posting datasets would likely improve both precision and coverage.

The gap between human-evaluated quality scores for summaries (88%) and bullet points (85%) reflects the greater structural constraint placed on the bullet generation task. The model must produce exactly three distinct bullets, each beginning with a past-tense action verb, within a token budget that may not accommodate all the information in a verbose raw description. The summary task has more degrees of freedom and benefits from the model’s stronger general capability for fluent prose.

One architectural consideration that warrants future investigation is the handling of very long work descriptions. The current implementation applies a fixed max_tokens limit, which may truncate output for candidates with extensive experience records. An adaptive token budget based on input length would produce more complete output in these cases.

VI. CONCLUSION

This paper has described an end-to-end AI resume generation system built around four cooperating modules: context-aware spell correction, domain knowledge graph-driven skill suggestion, and FLAN-T5-based zero-shot generation of professional summaries and bullet points, all integrated into a

Flask web application with ReportLab PDF rendering. Evaluation against a structured 100-record dataset demonstrates spell correction accuracy of approximately 88%, skill suggestion coverage of approximately 85%, and human-evaluated generation quality of approximately 88% for summaries and 85% for bullet points, with complete pipeline execution under 20 seconds on CPU hardware.

The system demonstrates that instruction-tuned encoder-decoder transformers are well-suited to professional document generation tasks when paired with targeted pre-processing and domain knowledge representations. The zero-shot approach removes the dependency on task-specific labelled training data, making the system straightforward to adapt to new professional domains by modifying prompts and extending the knowledge graph rather than retraining the model.

Future work will explore fine-tuning FLAN-T5 on curated resume datasets to improve output specificity, incorporating dynamically updated skill graphs derived from live job market data, and extending the template library to support industry-specific resume formats in domains such as academia and creative industries.

ACKNOWLEDGMENT

We would like to express our sincere gratitude to Dayananda Sagar University for providing the necessary infrastructure, computing resources, and a supportive learning environment that enabled us to successfully carry out this research work.

We also extend our appreciation to the developers and contributors of open-source technologies that played a crucial role in this project. In particular, we acknowledge the contributions of the Hugging Face Transformers library for enabling the use of the FLAN-T5 model, and the developers of PySpellChecker for efficient text correction.

These tools and frameworks were instrumental in the development and implementation of our AI-based Resume Generator system.

REFERENCES

- [1] E. Reiter and R. Dale, *Building Natural Language Generation Systems*. Cambridge, UK: Cambridge University Press, 2000.
- [2] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to Sequence Learning with Neural Networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27, 2014, pp. 3104–3112.
- [3] T. Brown *et al.*, "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.
- [4] H. W. Chung *et al.*, "Scaling Instruction-Finetuned Language Models," arXiv preprint arXiv:2210.11416, 2022. [Online]. Available: <https://arxiv.org/abs/2210.11416>
- [5] M. D. Yannakoudakis and R. Fawthrop, "An Intelligent Spelling Error Corrector," *Information Processing and Management*, vol. 19, no. 2, pp. 101–108, 1983.
- [6] M. Khauja, I. Kassou, and M. Ghogho, "A Survey on Text-Based Automatic Personality Prediction," *IEEE Access*, vol. 9, pp. 156706–156727, 2021.
- [7] ReportLab Inc., "ReportLab PDF Library User Guide," ReportLab, London, UK, 2023. [Online]. Available: <https://www.reportlab.com/documentation/>